

REPLICANT LAB · FICHA TÉCNICA DERIVADA DE MKDOCS

PULA

Arquitectura, despliegue, operación, seguridad y evidencia auditada.

Fuente

aplicaciones/pula.md

Versión reproducible

sha256:7131c8ed29ecc9c2f9ee11ed24a9ffc7efeda3c9ff7865ff4bd2e42d0514eaaa

PULA

Auditoría técnica y manual funcional de `ratienza/Pula` . El repositorio contiene dos líneas distintas: la etiqueta `v0.1-poc-beta` conserva la POC **Erasmushomes** para Bolonia/UNIBO, mientras que `main` implementa **Pula Apartment Automation** para alojamiento estudiantil en Pula, Croacia.

Estado comprobado

Elemento	Evidencia
Repositorio	<code>ratienza/Pula</code> · privado
<code>main</code> auditado	<code>ff8165cf7c9d1d4c5ae670c56486c25900a5754b</code>
POC congelada	<code>v0.1-poc-beta</code> · <code>6441ca79f9980e7179cbaf7378958d2628a3f80b</code>
Relación Git	Historias independientes; no existe ancestro común entre el tag y <code>main</code>
Checkout Nexus	<code>/opt/apps/Pula</code> · limpio y sincronizado; copia de consulta, no runtime validado
Producción	Cloud Run declarado por el proyecto; no inspeccionado ni modificado
Pruebas locales	TypeScript y build Vite/esbuild correctos sobre copia temporal
Tests automatizados	No existe suite; solo scripts exploratorios de scraping/Firebase
Datos versionados	<code>data.json</code> : 29 registros <code>new</code> con 29 direcciones de contacto

Calidad general: **5/10 para `main`** . La POC compila y concentra el flujo funcional, pero carece de tests, control de acceso, persistencia robusta y trazabilidad reproducible de Cloud Run. La etiqueta histórica obtiene **4/10** como prototipo: demuestra FastAPI/Gemini y el dossier, pero no es un producto endurecido.

Dos estados que no deben mezclarse

```
v0.1-poc-beta · ErasmusHomes · Bolonia / UNIBO
├─ FastAPI + SPA HTML
├─ Smart Deposit Audit · Gemini 1.5 Flash
├─ prototipo visual de cuatro pantallas
└─ dossier ReportLab de 12 páginas

main · Pula Apartment Automation · Pula / SCPU
├─ React 19 + Vite + Tailwind CSS 4
├─ Express 5 + TypeScript
├─ scraping SCPU + Gemini 2.5 Flash
├─ Gmail API
└─ persistencia local en data.json
```

El tag representa otra raíz Git. Los flujos de Bolonia se documentan como **POC histórica congelada**, no como capacidades del runtime actual.

Arquitectura de main

```

Navegador · React 19 / Vite / Firebase Auth
| REST /api/* · token Gmail cuando procede
▼
Express 5 · server.ts · 0.0.0.0:3000
├─ scraping SCPU con Cheerio
├─ extracción URL con Gemini 2.5 Flash
├─ envío y detección de respuestas con Gmail API
├─ cron 09:00 y 17:00 · Europe/Madrid
└─ lectura/escritura síncrona de data.json

```

El build ejecuta `vite build` y empaqueta `server.ts` con esbuild como `dist/server.cjs`. El repositorio no contiene Dockerfile, manifiesto Cloud Run, workflow CI ni definición de volumen persistente; Git no permite reproducir ni vincular el despliegue declarado con el SHA auditado.

Modelo de datos

Cada apartamento registra UUID, casero, superficie, capacidad, precio, ubicación, descripción, email, enlace, condiciones, contrato, estado y fecha. Puede añadir origen manual y datos de respuesta. El enlace funciona como clave de deduplicación.

Estados: `new`, `sent`, `following` y `discarded`.

Manual funcional de main

Descubrimiento automático

1. **Extraer Nuevos Pisos SCPU** o el cron inicia el proceso.
2. El servidor recorre hasta cinco páginas y limita el lote a 40 enlaces.
3. Cheerio extrae casero, email, precio, superficie, zona y contrato.
4. Se traducen expresiones croatas mediante sustituciones locales.
5. Se excluyen contratos detectados como anuales y se deduplica por URL.
6. El resultado se guarda y aparece en **Nuevas Oportunidades**.

La implementación contradice la afirmación documental “sin límite artificial”: hay un máximo de cinco páginas y 40 enlaces. Además, el scraper fija la capacidad como `Ver descripción`, aunque la regla de interfaz exige un número de personas.

Importación manual con IA

1. **Add URL**  abre el formulario.
2. El backend descarga la URL y elimina etiquetas ruidosas.
3. Envía hasta 12.000 caracteres a Gemini 2.5 Flash con esquema JSON.
4. Si Gemini no está disponible, aplica una heurística.
5. Marca la ficha como manual y deduplica por URL exacta.

No es segura para exposición pública hasta restringir protocolos, DNS/IP, redes privadas, redirecciones, tamaño y tiempo: hoy una petición anónima puede ordenar al servidor descargar una URL arbitraria, creando riesgo de SSRF.

Contacto y seguimiento Gmail

1. Firebase Google Sign-In entrega un access token conservado en memoria.
2. El backend construye un correo bilingüe inglés/croata y llama a Gmail API.
3. La comprobación revisa hasta 25 hilos y compara remitentes con caseros.
4. Una respuesta mueve un anuncio enviado a `following`.

Existe un defecto de integridad: el endpoint responde `success` y cambia el estado a `sent` incluso si falta un token utilizable o Gmail devuelve un error. “Enviado” no prueba que el correo haya salido.

Gestión visual

La SPA ofrece Nuevas Oportunidades, En Seguimiento, Enviados sin respuesta e Histórico, orden por fecha o precio y cambio manual de estado. Los botones de producto están presentes. `package.json` usa React 19, aunque la arquitectura escrita declara React 18.

POC histórica `v0.1-poc-beta`

Cuatro flujos visuales

Flujo	Cobertura comprobada
Onboarding UNIBO	Bolonia, Università di Bologna y fechas de movilidad
Discover Feed	Tarjetas de Via Zamboni y Santo Stefano
Mapa de seguridad y precios	Zamboni, Santo Stefano, Innerio y Saragozza
Ficha del piso	Precio, ubicación y detalle de Shared Apt. Zamboni

Son mockups y una SPA demostrativa; el tag no demuestra mapa real, base de datos, Gmail conectado ni transacciones de producción.

Smart Deposit Audit

FastAPI expone `POST /api/v1/audit-deposit` con dos imágenes y descripción. `GeminiService` intenta usar `gemini-1.5-flash` ; sin API o tras un fallo devuelve una respuesta simulada de devolución completa. Ese fallback no puede presentarse como dictamen real de IA.

No hay autenticación, límite de tamaño, validación MIME ni protección frente a carga abusiva. Los mensajes de excepción llegan al cliente y CORS permite cualquier origen junto con credenciales.

Dossier ReportLab

`Dossier_Oficial_ErasmusHomes_Definitivo_CERRADO.pdf` tiene 12 páginas, texto seleccionable y cuatro recursos visuales. Usa Letter, Helvetica, verde `#059669` , azul oscuro `#0F172A` y tablas de mercado, DAFO, precios y proyección financiera.

El generador contiene rutas absolutas a Google Drive, al directorio privado de Antigravity y a fuentes Windows, por lo que no es reproducible fuera del equipo original. El documento alterna las etiquetas v6.0 y v7.0. Sus cifras comerciales son hipótesis, no métricas validadas por código.

API REST observada

`main`

Método	Ruta	Función	Autenticación
GET	<code>/api/config</code>	Client ID OAuth	No
GET	<code>/api/apartments</code>	Lista completa	No
POST	<code>/api/apartments/scrape</code>	Scraping y escritura	No
POST	<code>/api/apartments/manual-url</code>	Descarga y analiza URL	No

Método	Ruta	Función	Autenticación
POST	/api/gmail/check-replies	Consulta Gmail y actualiza	Bearer de Google
PUT	/api/apartments/{id}/status	Modifica estado	No
POST	/api/apartments/{id}/send	Envía y marca <code>sent</code>	Bearer opcional en la práctica

No se observan OpenAPI versionado, rate limiting, sesión de aplicación, autorización por usuario ni validación estructural centralizada.

Etiqueta congelada

Método	Ruta	Función
GET	/health	Estado y disponibilidad Gemini
GET	/dossier-pdf	Descarga del dossier
GET	/	SPA demostrativa
POST	/api/v1/audit-deposit	Comparación fotográfica
## Riesgos y pendientes reales		

Prioridad alta

- Proteger lecturas y mutaciones con autenticación y autorización.
- Bloquear SSRF en la importación manual.
- Sacar `data.json` y los contactos del repositorio; tratar los emails como datos personales.
- No marcar `sent` tras error o ausencia de Gmail.
- Definir persistencia compatible con Cloud Run y operaciones concurrentes.

Prioridad media

- Usar bloqueo y reemplazo atómico en vez de escrituras síncronas directas.
- Restringir CORS, limitar cuerpos y cargas, añadir rate limiting y cabeceras.
- Validar HTTP del scraper, fijar timeouts y evitar logs OAuth detallados.
- Añadir tests unitarios, API, scraper con fixtures, Gmail simulado y CI.
- Elegir y fijar un único gestor de dependencias y lockfile.

Deriva documental

- Arquitectura: React 18; manifiesto: React 19.
- Documentación: scraping ilimitado; código: cinco páginas y 40 enlaces.
- `AGENTS.md` reserva Gmail y URL manual para v2.0; ambos ya están en `main`.
- Regla visual: prohíbe `Ver descripción`; el scraper lo asigna a `size`.
- Cloud Run y Nginx declarados sin Dockerfile ni configuración reproducible.
- La automatización Python usa mock, OAuth local y `TEST_MODE = True`; no pertenece al runtime Express demostrado.

Pruebas y límites

- ☒ Checkout Nexus limpio y sincronizado.
- ☒ Tag resuelto al commit esperado.
- ☒ Instalación temporal, `tsc --noEmit` y build Vite/esbuild.

-  Sintaxis de los módulos Python históricos.
-  No existe suite automatizada.
-  No se llamó a SCPU, Gemini, Gmail, Firebase ni datos vivos.
-  Cloud Run no fue inspeccionado ni validado.
-  El PDF se comprobó y renderizó estructuralmente; no se declara revisión editorial completa.

Operación futura y traslado

Antes de DigitalOcean se necesitan runtime reproducible, almacenamiento persistente, secretos fuera de Git, autenticación, backup/restore, healthcheck y tests. No deben reutilizarse datos o credenciales de Cloud Run implícitamente.

Checkout  Runtime : PULA no se desplegó en Nexus. Cloud Run no se tocó.